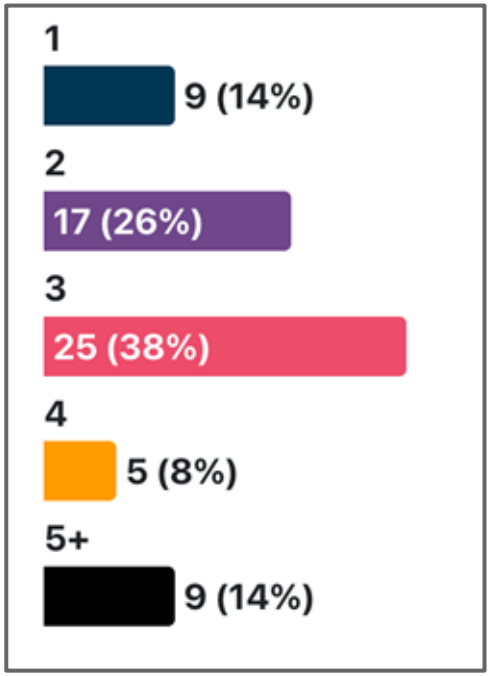
what Q did I ask last semester to get this result?



clicker.cs.illinois.edu

Q1

~Code~
101

CS 340

AIOHTTP



Updates

1. Everything is graded – on canvas
2. MP 7 - aiohttp lite, due next week tuesday
3. HW 8 out

AIOHTTP

Today’s LGs - Be able to make simple servers with AIOHTTP

Agenda

1. Review of HTTP
2. AIOHTTP
3. Function Decorator Review

Layers of a packet Review

Layer	Protocols	Use
Application	HTTP, SSH, SMTP, etc.	programmer-visible messages
Transport	**TCP, UDP**	reach a specific program
Internet	**IP**, which has two versions: IPv4 and IPv6	reach a specific computer

HTTP

Request Methods

client

GET - asks for a resource

POST - sends data to the server

PUT - sends data to the server (overwrites existing)

DELETE - removes a resource

Responses

server

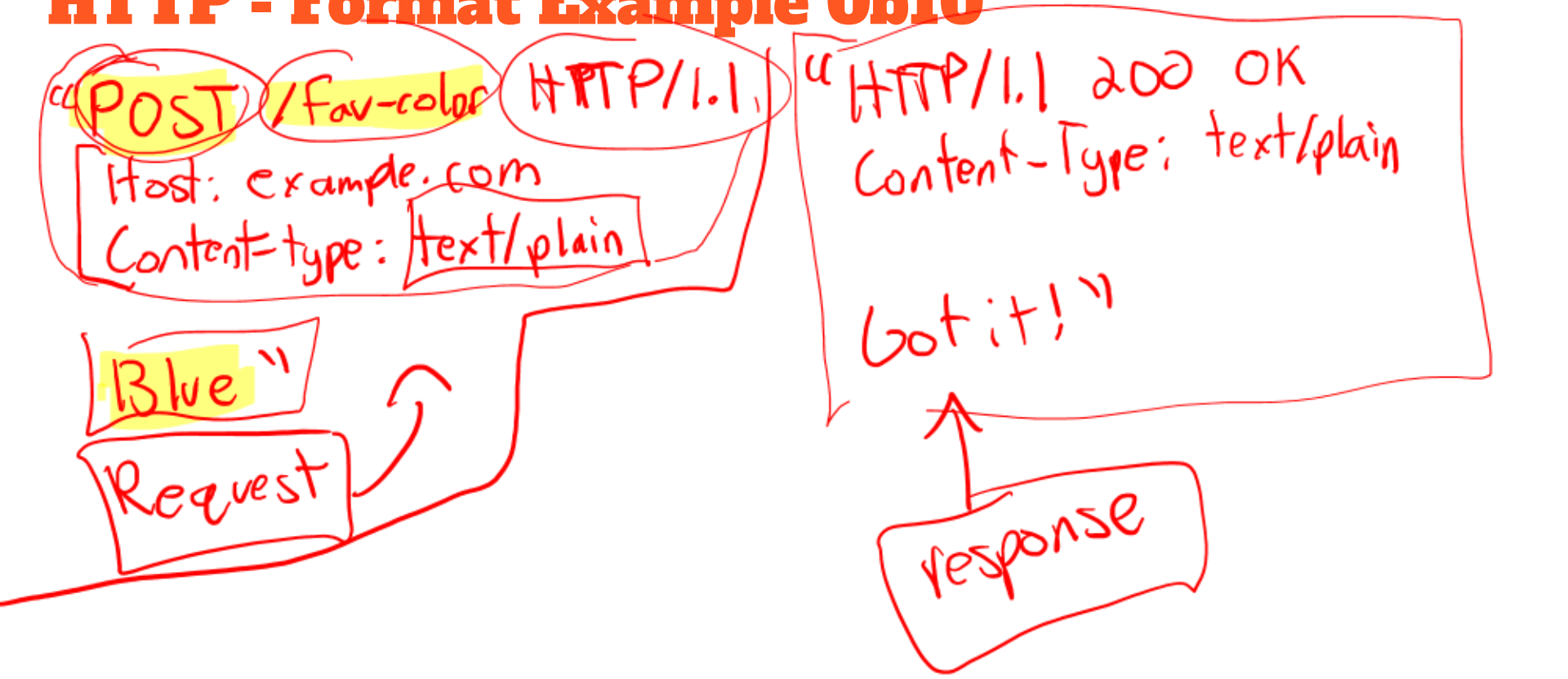
200 - ok

400 - bad request

404 - not found

500 - internal Server Error

HTTP - Format Example 0b10

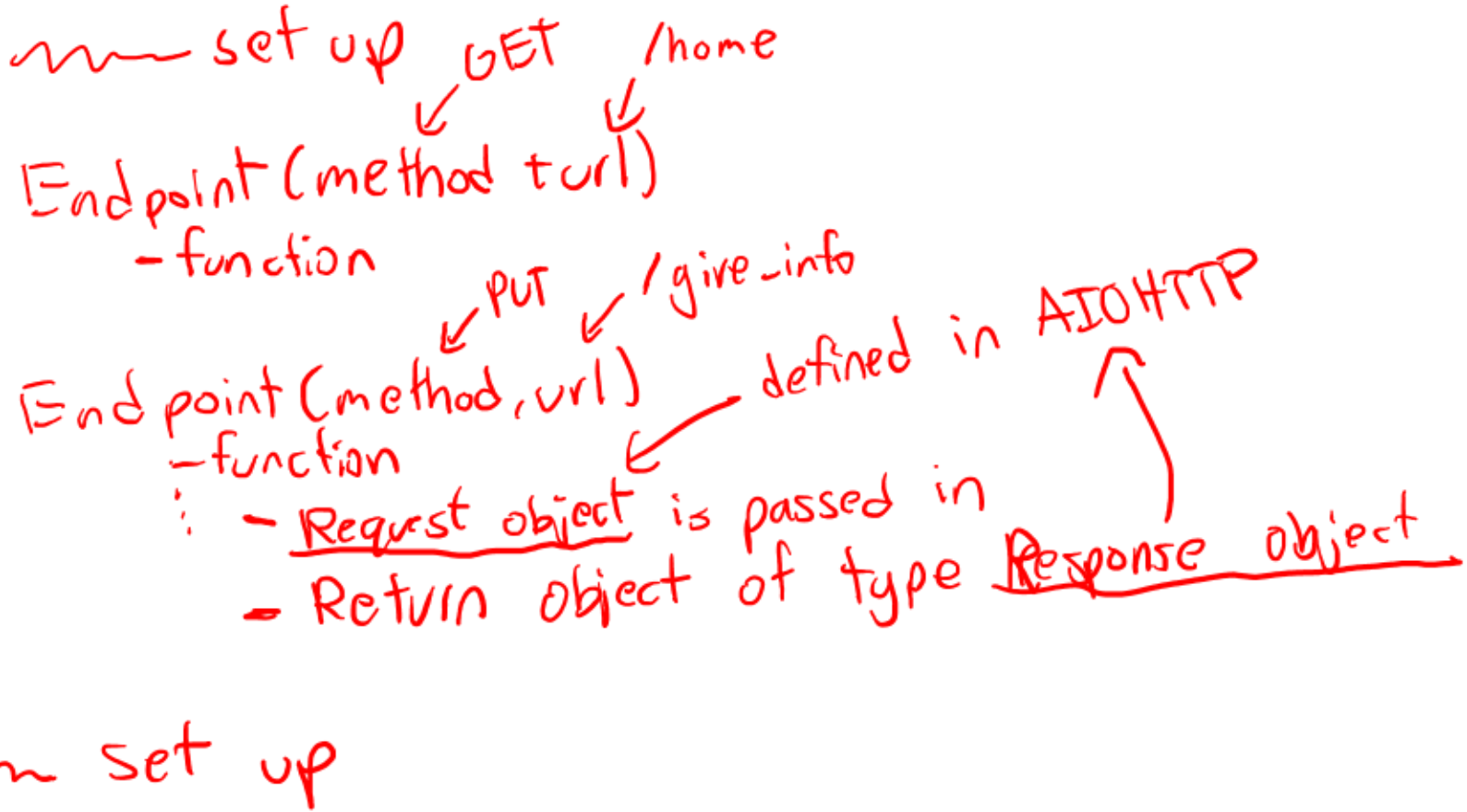


Welcome to AIOHTTP

Asynchronous HTTP Client/Server for asyncio and Python.

Python Library that handles raw bytes of HTTP Requests and Responses

Writing a server with AIOHTTP



AIOHTTP Server Example

```python
from aiohttp.web import RouteTableDef, Application, Request, Response, run_app
#import web <-- for mp7

routes = RouteTableDef()

@routes.get("/")
async def get_homepage(request: Request) -> Response:
    #Get data from http request
    #do some logic
    #create and retunr response object

@routes.put("/other")
async def add_to_data(request: Request) -> Response:
    #Get data from http request
    #do some logic
    #create and retunr response object


if __name__ == '__main__':
    app = Application()
    app.add_routes(routes)
    run_app(app, host='127.0.0.1', port=4582)
```

Getting Data from a Request Object

functions

```
async text(self) -> str
    Return BODY as text using encoding from .charset.
```

```
async json(self, *, loads: Callable[[str], Any] = <function loads at 0x7f8845f885e0>) -> Any
    Return BODY as JSON.
```

inside endpoint function

```
text = await request.text()
dict = await request.json()
```

Async

Idea - async let's the program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

- Use the await keyword to indicate the program can switch if its waiting.
- await needs to be used to invoke another async function (coroutine)

AIOHTTP Creating a Response ← class

constructor

```
__init__(self, *, body: Any = None, status: int = 200, reason: Optional[str] = None, text: Optional[str] = None, headers: Union[Mapping[Union[str, multidict._multidict.is
    Initialize self.  See help(type(self)) for accurate signature.
```

return Response(text="Not found", status=404)

return Response(text=json.dumps({"name": "Alice"}),
content_type="application/json")

return Response(text=open("index.html").read(),
content_type="text/html")

Example Endpoints

```
23  @routes.get("/")
24  async def get_homepage(request: Request) -> Response:
25      return Response(text=open("index.html").read(),
26                      content_type="text/html")
27
28  possiblities = {"potato":0, "radish": 0, "pumpkin": 0}
29
30  @routes.put("/other")
31  async def add_to_data(request: Request) -> Response:
32      global possiblities
33      text = await request.text()
34      if text not in possiblities:
35          return Response(text="Wrong input", status=400)
36      else:
37          possiblities[text] += 1
38          return Response(text=json.dumps(possiblities),
39                          content_type="application/json")
40
```

What endpoint(s) does this server support?

clicker.cs.illinois.edu

Q2

~Code~
101



GET /

```python
@routes.get("/")
async def get_homepage(request: Request) -> Response:
    return Response(text=open("index.html").read(),
                    content_type="text/html")

possiblities = {"potato":0, "radish": 0, "pumpkin": 0}

@routes.put("/other")
async def add_to_data(request: Request) -> Response:
    global possiblities
    text = await request.text()
    if text not in possiblities:
        return Response(text="Wrong input", status=400)
    else:
        possiblities[text] += 1
        return Response(text=json.dumps(possiblities),
                        content_type="application/json")
```

PUT /other

2 endpoints

What would happen to line 37 if I commented out line 32?

```
23  @routes.get("/")
24  async def get_homepage(request: Request) -> Response:
25      return Response(text=open("index.html").read(),
26                      content_type="text/html")
27
28  possiblities = {"potato":0, "radish": 0, "pumpkin": 0}
29
30  @routes.put("/other")
31  async def add_to_data(request: Request) -> Response:
32      global possiblities
33      text = await request.text()
34      if text not in possiblities:
35          return Response(text="Wrong input", status=400)
36      else:
37          possiblities[text] += 1
38          return Response(text=json.dumps(possiblities),
39                          content_type="application/json")
40
```

error

clicker.cs.illinois.edu

Q3

~Code~ 101

What type is the variable "text" referencing?

```
23  @routes.get("/")
24  async def get_homepage(request: Request) -> Response:
25      return Response(text=open("index.html").read(),
26                      content_type="text/html")
27
28  possiblities = {"potato":0, "radish": 0, "pumpkin": 0}
29
30  @routes.put("/other")
31  async def add_to_data(request: Request) -> Response:
32      global possiblities
33      text = await request.text()
34      if text not in possiblities:
35          return Response(text="Wrong input", status=400)
36      else:
37          possiblities[text] += 1
38          return Response(text=json.dumps(possiblities),
39                          content_type="application/json")
40
```

string

status 200 default

clicker.cs.illinois.edu

Q4

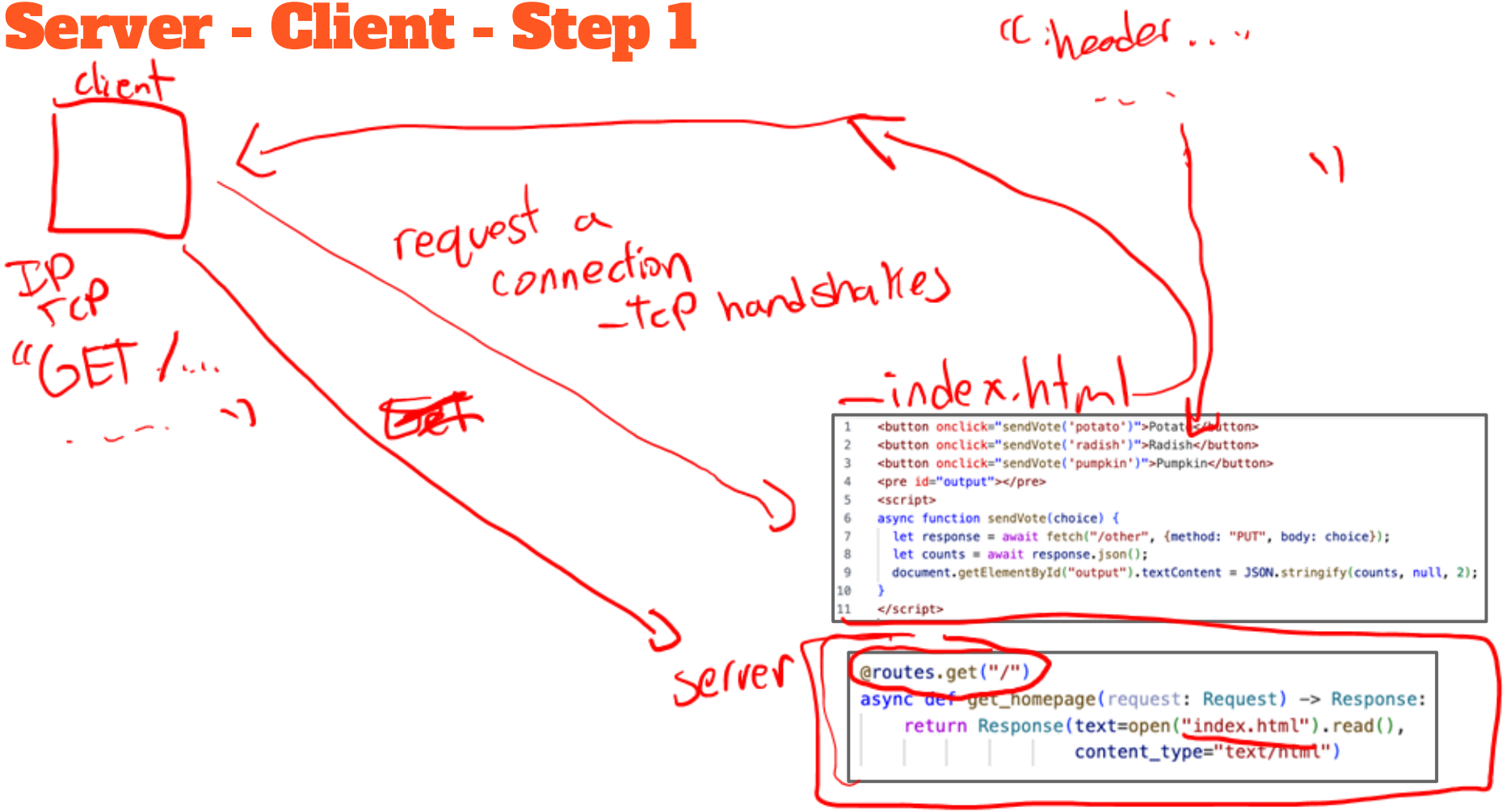
~Code~
101

Example index.html -not on the exam

```
1   <button onclick="sendVote('potato')">Potato</button>
2   <button onclick="sendVote('radish')">Radish</button>
3   <button onclick="sendVote('pumpkin')">Pumpkin</button>
4   <pre id="output"></pre>
5   <script>
6   async function sendVote(choice) {
7     let response = await fetch('/other', {method: "PUT", body: choice});
8     let counts = await response.json();
9     document.getElementById("output").textContent = JSON.stringify(counts, null, 2);
10  }
11  </script>
```

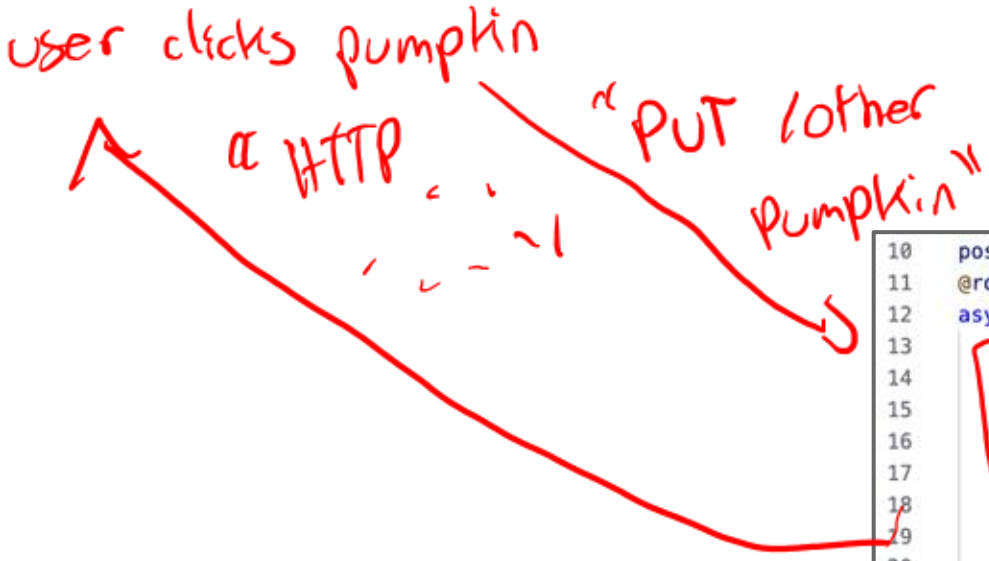


Server - Client - Step 1



Server - Client - Step 2

```html
<button onclick="sendVote('potato')">Potato</button>
<button onclick="sendVote('radish')">Radish</button>
<button onclick="sendVote('pumpkin')">Pumpkin</button>
<pre id="output"></pre>
<script>
async function sendVote(choice) {
  let response = await fetch("/other", {method: "PUT", body: choice});
  let counts = await response.json();
  document.getElementById("output").textContent = JSON.stringify(counts, null, 2);
}
</script>
```




```python
possiblities = {"potato":0, "radish": 0, "pumpkin": 0}
@routes.put("/other")
async def add_to_data(request: Request) -> Response:
    global possiblities
    text = await request.text()
    if text not in possiblities:
        return Response(text="Wrong input", status=400)
    else:
        possiblities[text] += 1
        return Response(text=json.dumps(possiblities),
                        content_type="application/json")
```

Running a Server

```
18  if __name__ == '__main__':
19      app = Application()
20      app.add_routes(routes)
21      run_app(app, host='0.0.0.0', port=4582)
```

specifies which IP address to listen at

ex) 127.0.0.1 ← localhost ← loop back

Private address ← from your LAN

Public IP ← if machine is directly on the internet

Demo

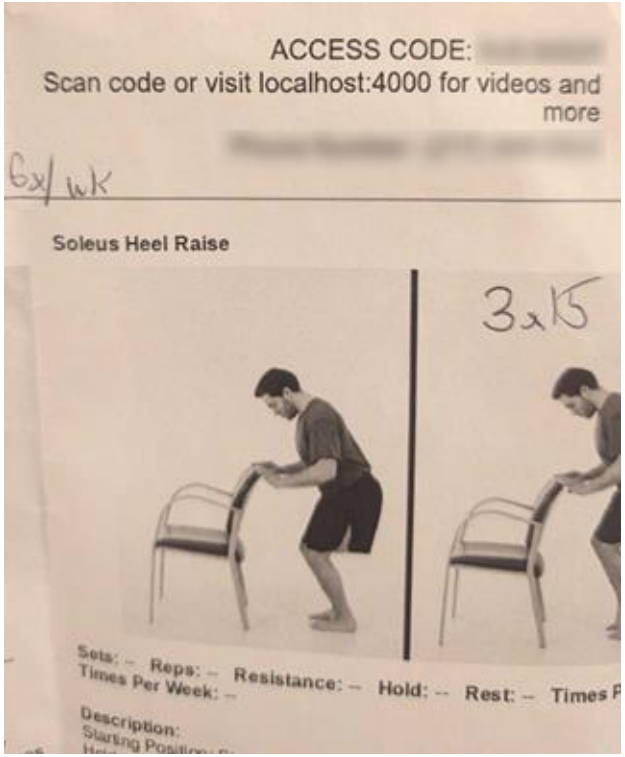
Summary

Test your server locally

- localhost:PORT
- curl
  - -X method
  - -v verbose
  - -d body of the request
  - -H content type of body

```
curl -X POST http://localhost:34643/items -H "Content-Type: application/json" -d '{"name": "test"}'
```

Real World Implications: How would you explain to the PT the issue? Explain to a partner.



Agenda

1. Review of HTTP

1. AIOHTTP

1. **Function Decorator Review**

What prints?

```
1   x = 4
2   def thing1():
3       x = 20
4       return x
5
6   def thing2():
7       x = -20
8       return x
9
10  if x > 0:
11      winner = thing1()
12  else:
13      winner = thing2()
14  print(winner)
```

clicker.cs.illinois.edu

Q5

~Code~
101



What prints?

```
1    x = 4
2    def thing1():
3        x = 20
4        return x
5
6    def thing2():
7        x = -20
8        return x
9
10   if x > 0:
11       winner = thing1
12   else:
13       winner = thing2
14   print(winner)
```

clicker.cs.illinois.edu

Q6

~Code~
101

What prints?

```
1    def thing1():
2        x = 20
3        return x
4
5    def thing2(var):
6        x = -20 + var
7        return x
8
9    winner = thing2
10   print(winner(thing1()))
```



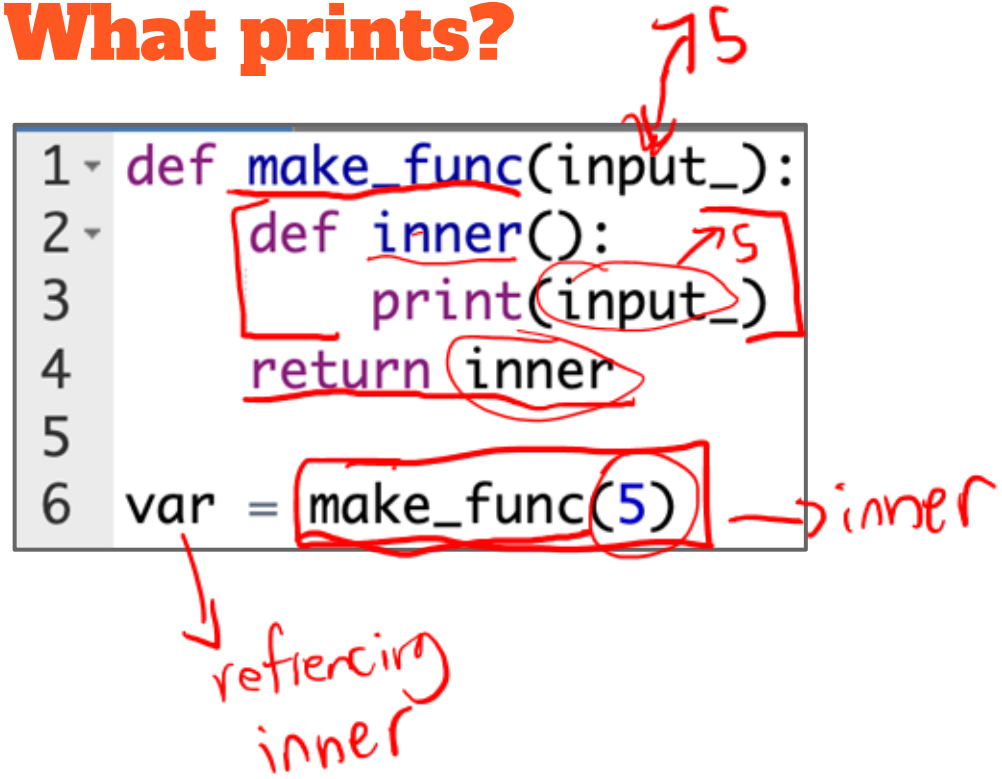
clicker.cs.illinois.edu

Q7

~Code~
101

What prints?

```
1 def make_func(input_):
2     def inner():
3         print(input_)
4     return inner
5
6 var = make_func(5)
```



clicker.cs.illinois.edu

Q8

~Code~
101

What prints?

```
1 def make_func(input_):
2     def inner():
3         print(input_)
4     return inner
5
6 var = make_func(5)()
```

print 5

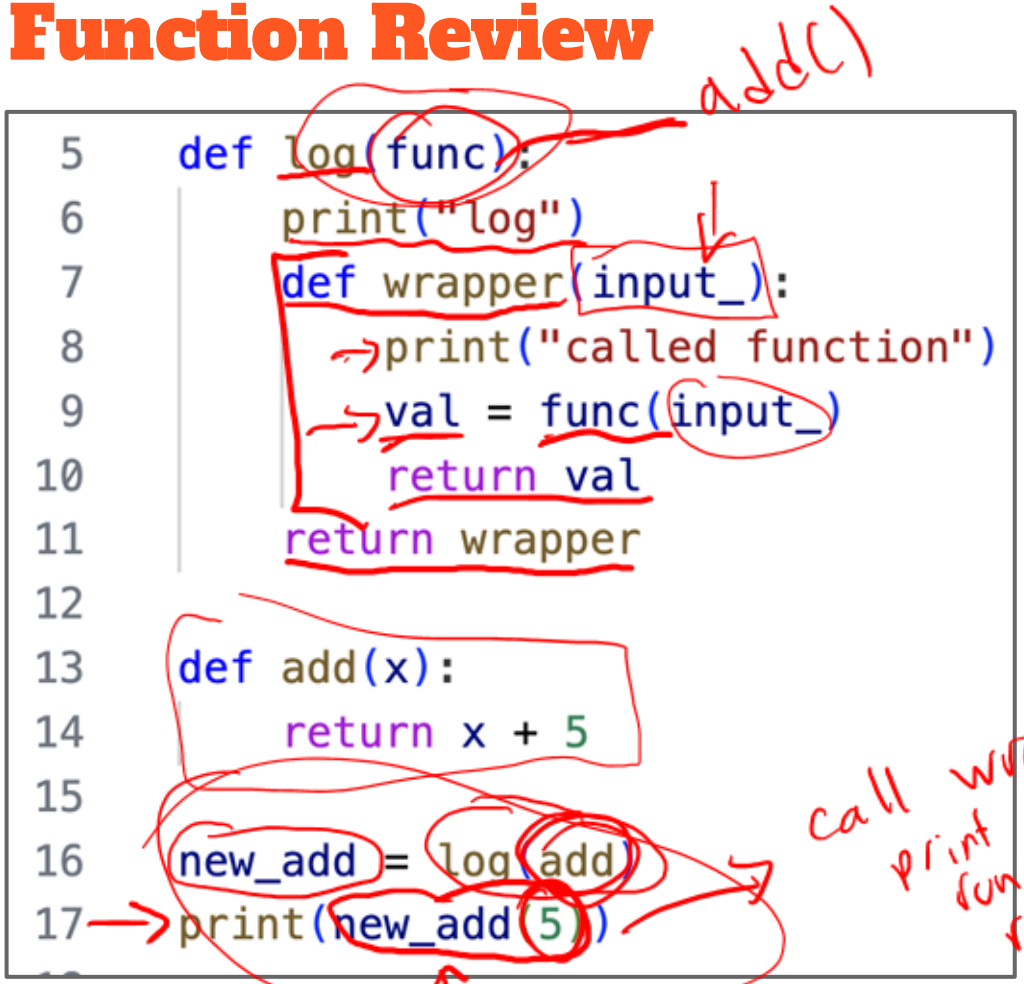
clicker.cs.illinois.edu

Q9

~Code~
101

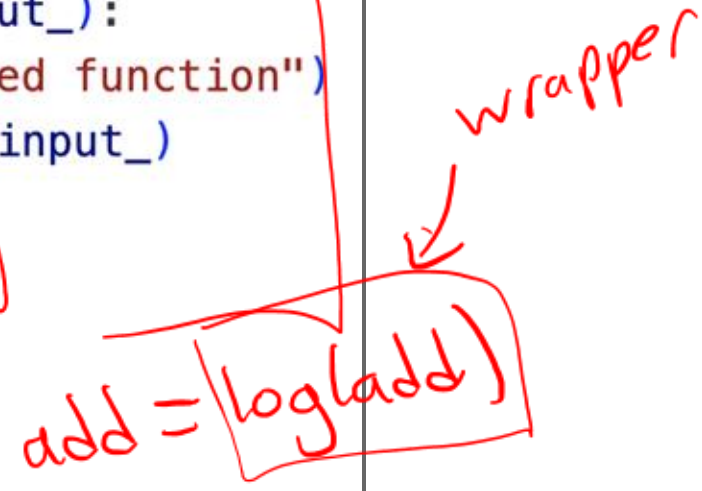
Function Review

```
 5    def log(func):
 6        print("log")
 7        def wrapper(input_):
 8            print("called function")
 9            val = func(input_)
10            return val
11        return wrapper
12
13    def add(x):
14        return x + 5
15
16    new_add = log(add)
17    print(new_add(5))
```

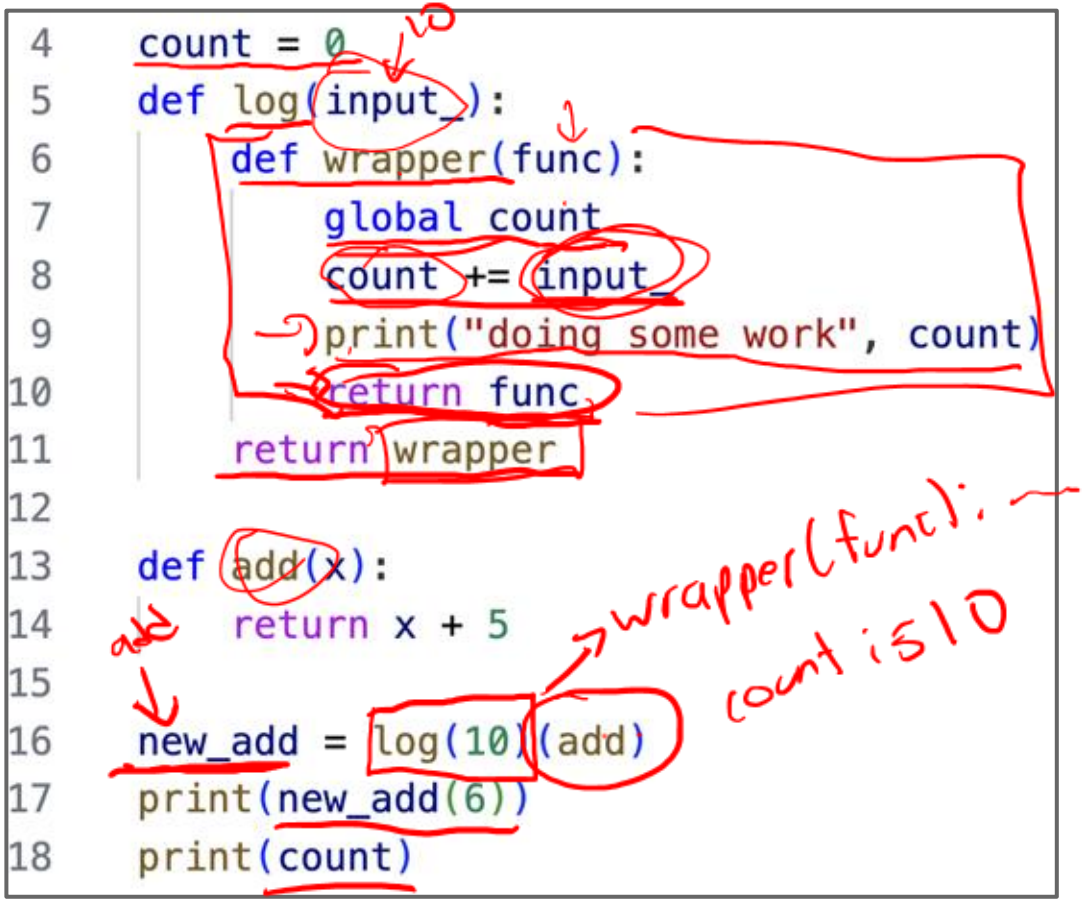


Function Review - with decorator

```
5   def log(func):
6       print("log")
7       def wrapper(input_):
8           print("called function")
9           val = func(input_)
10          return val
11      return wrapper
12
13  @log
14  def add(x):
15      return x + 5
16
17  print(add(5))
```



Function Review



Function Review - with decorator

```
4    count = 0
5    def log(input_):
6        def wrapper(func):
7            global count
8            count += input_
9            print("doing some work", count)
10           return func
11       return wrapper
12
13   @log(10)
14   def add(x):
15       return x + 5
16
17   print(add(5))
```

add = log(10)(add)

```python
routes = web.RouteTableDef()

@routes.get("/hello")
async def hello(request):
    return web.Response(text="Hello!")
```

function call

hello = routes.get("/hello")(hello)

```python
class RouteTableDef:
    def __init__(self):
        self.routes = {}

    def get(self, path):
        def decorator(func):
            self.routes[("GET", path)] = func
            return func
        return decorator
```

Feedback!

https://forms.gle/vTw8Cws5TMmWLSpJ9

